QCLAIRVOYANCE QUANTUM LABS
Rewriting Reality With Quantum Technology

Where is our cryptography?

Hands-on cryptographic discovery and CBOM generation for post-quantum migration

Preet Raj Gupta

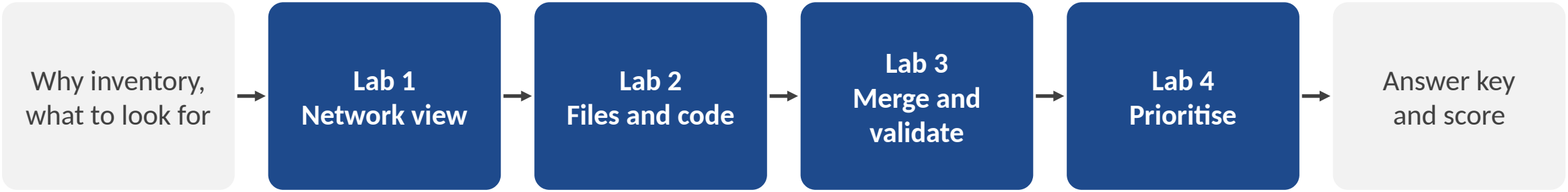
Cryptography Analyst, QClairvoyance Quantum Labs

ISEA-ISAP 2026 · Workshop



Outline

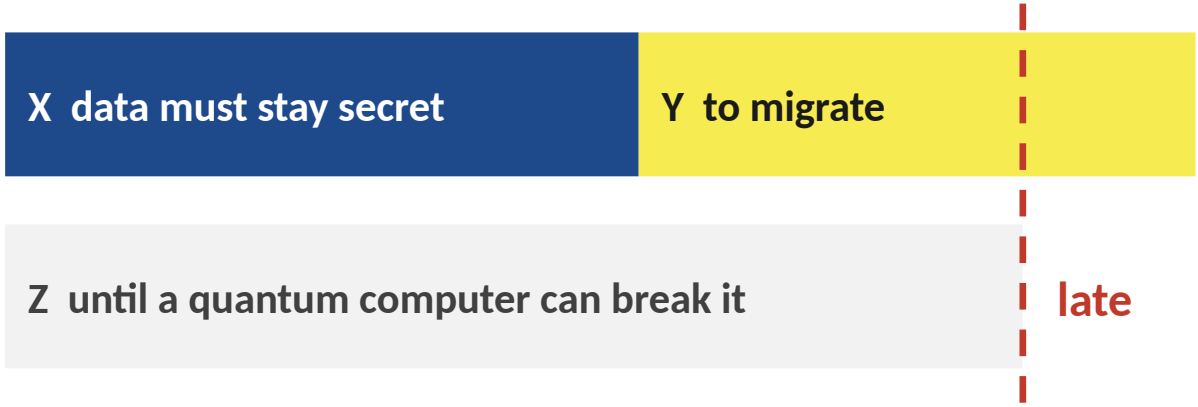
Where is our cryptography, *and how do we know we have found all of it?*



Work in pairs on BankLite, a small synthetic payment system with 13 cryptographic assets planted in it.

Inventory comes before migration

Mosca's inequality



If X + Y > Z, data recorded today is exposed before migration finishes. Y can only be estimated for cryptography that has been found.

What India's guidance asks for

DST / NQM Roadmap to Quantum Resiliency (May 2026)

Inventory is the first step of Milestone 1, and CBOMs from vendors are mandated from FY 2027-28.

CERT-In Technical Guidelines v2.0

Define the minimum elements a CBOM must carry (Tables 8 and 9).

Key exchange has moved; certificates have not

Dubey et al., IIT Jammu, 2026: top sites plus Indian bank and government domains

49%

support hybrid ML-KEM key exchange

0%

use post-quantum certificates

An inventory has to record key exchange and authentication separately, for every system.

Where inventories stall in practice

30%

of organisations have completed a cryptographic inventory

IBM IBV survey, 2025

~40%

of assets had no clear owner after a 14-month discovery programme

KU Leuven COSIC case study, 2026

What got in the way

Ownership Infrastructure, application team or vendor: nobody was sure.

Black boxes SaaS and payment platforms kept their cryptography behind contracts.

Legacy Old systems gave only partial or indirect evidence.

Operations Scanning had to fit production change windows.

IBM IBV, Secure the post-quantum future, Oct 2025, p. 12 · Zelenovic et al., arXiv 2605.16549, pp. 14-15

What counts as a cryptographic asset?

The four asset types in CycloneDX (ECMA-424), with BankLite examples

`algorithm`

A named primitive and its parameters

BankLite: ML-KEM-768, SHA-1, AES-128-ECB, HMAC-SHA256

`certificate`

Subject, issuer, validity and the algorithms it uses

BankLite: api.banklite.local, ECDSA P-256 with SHA-256

`protocol`

A protocol, its version and the cipher suites it offers

BankLite: TLS 1.2 on :8444 still accepting AES128-SHA

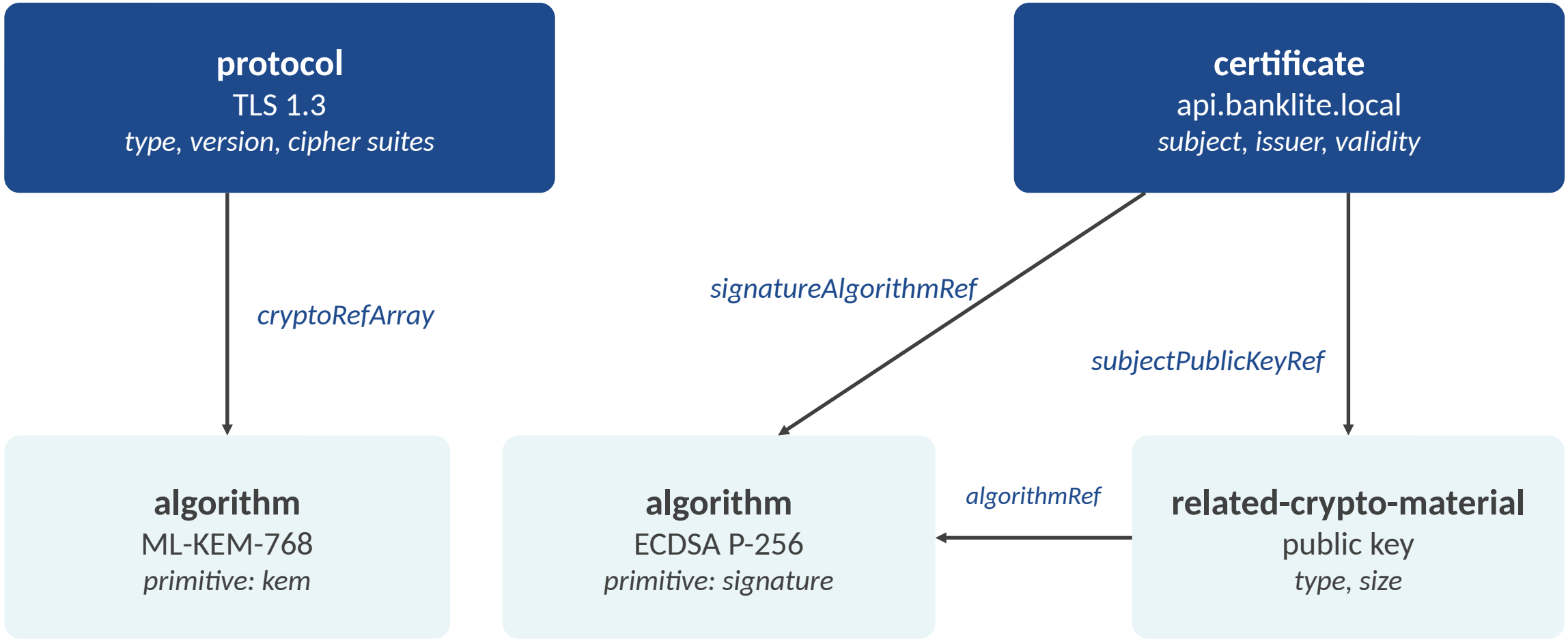
`related-crypto-material`

Keys, secrets, keystores and other material

BankLite: an unencrypted RSA-2048 private key backup

A CBOM is a graph

One BankLite TLS endpoint, as CycloneDX models it



Three views of cryptography

Each answers a different question and has its own blind spot

Wire live handshake	Files certificates, keys, config	Code source code
Answers **What is negotiated?**	Answers **What is deployed?**	Answers **What did the developer intend?**
Cannot see Anything that never crosses the network	Cannot see Whether it is ever used	Cannot see Choices made in config or at runtime
Lab 1: openssl s_client, testssl.sh	**Lab 2: CBOMKit-theia**	**Lab 2: cdxgen**

BankLite

A small synthetic payment system with 13 planted cryptographic assets

Payments API :8443
Node.js, TLS 1.3, hybrid key exchange

→ `certs/`
server certificates and keys

Legacy endpoint :8444
TLS 1.2 for an old acquirer link

→ `keystore/` `keys/`
PKCS#12 keystore, a key backup

Settlement signer
Java, signs the daily settlement file

→ `config/` `src/`
OpenSSL policy, TLS settings, JS code

13

planted assets

Offline, in one container. No real keys or data.

Setup

Linux: Docker Engine · Windows 10/11: Docker Desktop with WSL2 (see docs/WINDOWS.md)

```
# once, before the session (needs internet)
git clone https://github.com/preet-qclair/banklite-lab && cd banklite-lab
docker build -t banklite-lab .

# in the room (offline)
mkdir -p out
docker run --rm -it --user "$(id -u):$(id -g)" \
  -v "$PWD/out:/lab/out" banklite-lab ./check.sh
```

github.com/preet-qclair/banklite-lab

Look for: `ALL OK` then start the lab shell without ./check.sh

Laptop: 4 GB RAM, 3 GB free disk.
Windows PowerShell: leave out --user.

If Docker fails: pair with a neighbour. Reference outputs for every step are in `solutions/`, so Labs 3 and 4 work without Docker.

THE LAB PATH

Network
OpenSSL, testssl.sh

Files
CBOMKit-theia

Code
cdxgen

Merge
CycloneDX CLI, jq

Prioritise
worksheet

LAB 1

Lab 1: what is negotiated on the wire?

```
# 8443: offer the hybrid group first
openssl s_client -connect localhost:8443 \
  -groups X25519MLKEM768:X25519 </dev/null \
  | grep -E 'Negotiated|Cipher is|Peer sig'

# 8444: full scan
testssl --quiet -p -P -f -S localhost:8444
```

Then write a CBOM by hand: copy templates/network.json and replace the four FILL_ME values.

Questions

- Which port protects recorded traffic against a future quantum computer, and why?
- What does 8443 fall back to when the client has no ML-KEM?
- 8444 offers AES128-SHA. What key exchange is that, and why is it the worst case for harvest-now, decrypt-later?

THE LAB PATH

- **Network** — OpenSSL, testssl.sh
- **Files** — CBOMKit-theia
- **Code** — cdxgen
- **Merge** — CycloneDX CLI, jq
- **Prioritise** — worksheet

LAB 2

Lab 2: what is deployed, and what was intended?

```
# files: certificates, keys, keystores, config
cbomkit-theia dir /banklite > out/files.json

# source code: crypto API calls
cdxgen --include-crypto --no-install-deps \
  -o out/code.json /banklite
```

```
# then read the code yourself
ls -R /banklite
cat /banklite/src/*.js /banklite/settlement/*.java
```

Do not pass -t js to cdxgen: with a single -t, its crypto collector does not run.

Questions

- Which assets did both tools find, and which did only one find?
- Did anything get the wrong asset type? Look at what cdxgen calls backup_rsa.pem.
- Find one asset in the source code that neither tool reported.

THE LAB PATH

- **Network** OpenSSL, testssl.sh
- **Files** CBOMKit-theia
- **Code** cdxgen
- **Merge** CycloneDX CLI, jq
- **Prioritise** worksheet

LAB 3

Lab 3: merge and validate

1 Merge

```
cyclonedx merge --input-files
  network.json files.json
code.json
```

2 Validate

```
cyclonedx validate ... --input-
version v1_7
```

3 It fails

```
merge writes "state": "null"
the schema rejects it
```

4 Repair, validate again

```
jq 'walk(... del(.state) ...)'
```

Questions

- Do the links between a certificate, its key and its algorithm survive the merge?
- An auditor gets only the merged CBOM. Which of your findings are missing from it?

Validate every CBOM, including tool output.

THE LAB PATH

- **Network** — OpenSSL, testssl.sh
- **Files** — CBOMKit-theia
- **Code** — cdxgen
- **Merge** — CycloneDX CLI, jq
- **Prioritise** — worksheet

LAB 4

Lab 4: prioritise

Mosca ratio = (X + Y) / Z

Z = 10 years in the worksheet. Above 1.0 means late.
Priority = ratio × exposure (1 to 3)

For each asset, fill in

- Does a quantum computer break it? Shor breaks RSA, ECDSA and ECDH. Grover only weakens AES and SHA-2.
- X: years it must stay secret or valid. Y: years to migrate it.
- Exposure: who can reach it. 1 internal, 2 partner, 3 internet.

Then decide

1. Pick the top five to migrate first.
2. Mark every asset that needs a classical fix today, quantum or not.
3. Find one asset that should not be migrated at all.

worksheet/prioritise.xlsx · opens in Excel, LibreOffice or Google Sheets

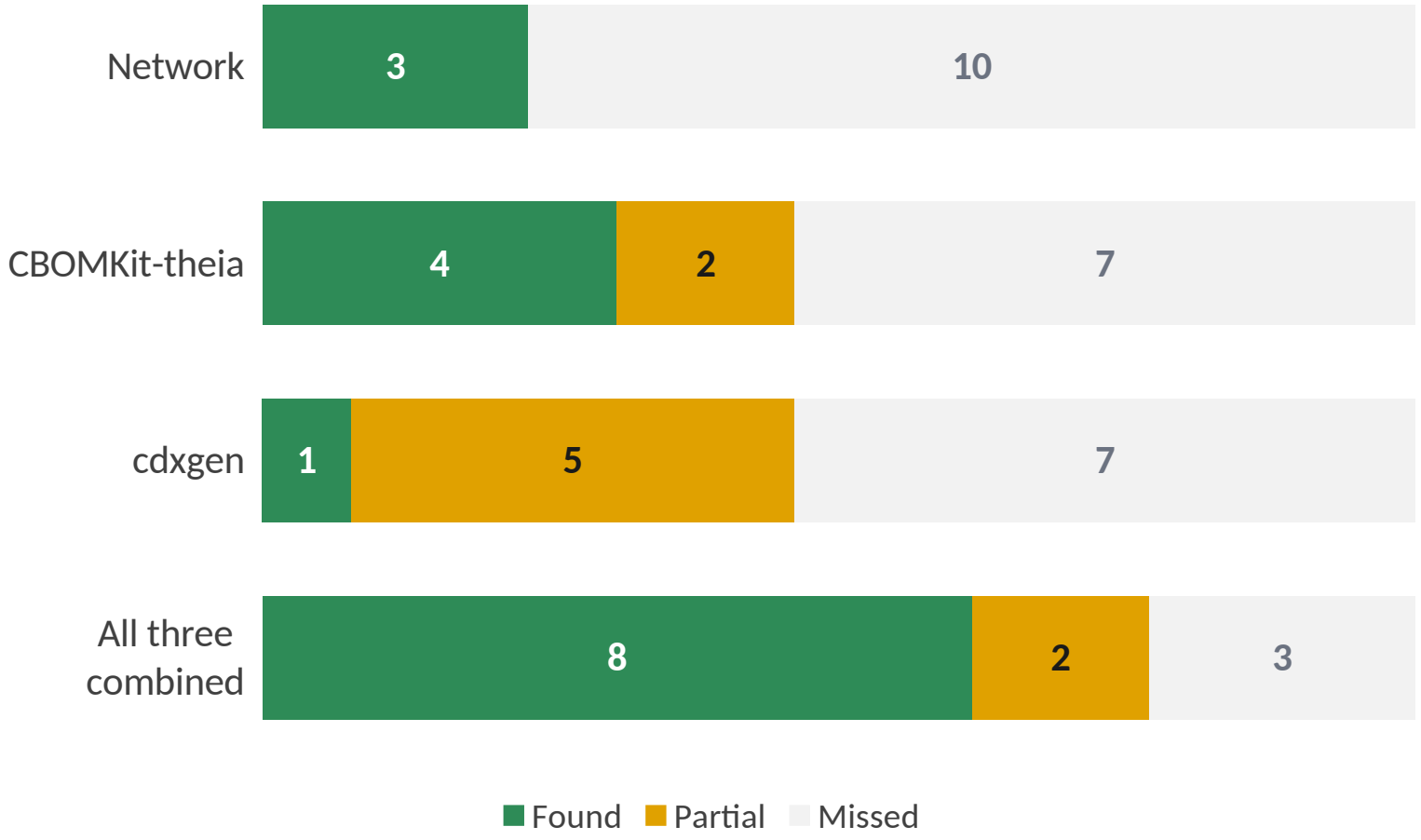
Answer key

Scorer output on the pinned lab image

Asset	What	Network	theia	cdxgen	All three
A01	TLS 1.3 endpoint :8443	found	-	-	found
A02	ML-KEM-768 in X25519MLKEM768	found	-	-	found
A03	TLS 1.2 :8444 with AES128-SHA	found	partial	-	found
A04	ECDSA P-256 cert api.banklite.local	-	found	partial	found
A05	RSA-2048 cert legacy.banklite.local	-	found	partial	found
A06	Unencrypted RSA key backup	-	found	partial	found
A07	AES-128-ECB, hard-coded key	-	-	-	-
A08	SHA-1 receipt IDs	-	-	found	found
A09	HMAC-SHA256 webhooks	-	-	partial	partial
A10	PKCS#12 settlement keystore	-	partial	partial	partial
A11	SHA256withRSA signing (Java)	-	-	-	-
A12	OpenSSL policy, SECLEVEL=1	-	found	-	found
A13	AES-256-CBC, split key, zero IV	-	-	-	-

Coverage by view

Assets found per view, out of 13



4 / 13

best single view

8 / 13

all three together

3

missed by every tool

Why three assets were missed

From a fixable bug to a fundamental limit

1 A tool bug

A07 AES-128-ECB in card.js
cdxgen detects the cipher, then drops it while normalising its name.

Fix: report it upstream

2 A missing view

A11 SHA256withRSA in SettlementSigner.java
No tool in the lab scans Java source code.

Fix: add a scanner for that language

3 A runtime choice

A13 AES-256-CBC in vault.js
The cipher name is assembled at runtime, from pieces.

No static tool can be sound and complete here in general (Rice's theorem)

easy to fix → fundamental

Bugs in the tools

Found while building this lab; reproducible from solutions/

Keys called certificates

cdxgen labels three private keys and a PKCS#12 keystore as certificate.

A wrong identifier

cdxgen gives SHA-1 an OID from Novell's branch, not SHA-1's 1.3.14.3.2.26.

Detected, then dropped

cdxgen finds aes-128-ecb, then loses it while normalising the name.

A silent switch

Pass -t js to cdxgen and its crypto collector does not run: zero crypto components.

PQC config invisible

CBOMKit-theia ignores the Groups line in openssl.cnf, so X25519MLKEM768 never appears.

The merge breaks the schema

CycloneDX CLI writes "state": "null" six times, and the result fails validation.

Published studies report the same gaps

Tools disagree

CBOM generators run on the same repositories rarely reported the same assets.
Bögli et al., ICPC 2026

Recall is limited

A recent static scanner recovered about two-thirds of the planted occurrences in its own benchmark.
Näther and Hirsch, 2026

Hybrids are misread

Naive scanners report the classical half of a hybrid as a migration target.
Yamamuro et al., ICISSP 2026

Binaries hide it

Name-based scanning finds nothing once a binary is stripped.
Junaid, 2026 preprint

Report an inventory's coverage alongside its findings.

Not everything needs migration

The 13 BankLite assets in three groups (A06 belongs in two)

Quantum-vulnerable

- A03 TLS 1.2 static RSA key transport
- A04 ECDSA P-256 certificate
- A05 RSA-2048 certificate
- A06 RSA key backup
- A10 RSA settlement keystore
- A11 SHA256withRSA signing
- A12 policy that re-enables A03

Classically broken today

- A07 AES-ECB, hard-coded key
- A08 SHA-1 receipt IDs
- A13 zero IV, embedded key
- A06 unencrypted key on disk

Fine as it is

- A02 ML-KEM-768 (the target)
- A09 HMAC-SHA256
- A01 hybrid TLS 1.3 key exchange

Migrate first: A03, then A10 and A11 together, and A12. Fixing A12, one line of OpenSSL policy, also removes A03 on every service on the host.

Open problems

1

Ground truth

Realistic benchmarks across languages, configuration and deployment. Current sets, such as Cryben and BankLite, are small and synthetic.

2

Matching across tools

Deciding when two CBOMs describe the same asset, so that tool outputs can be merged without duplicates or losses.

3

Runtime cryptography

Algorithms chosen at runtime cannot be resolved statically; combining static and dynamic analysis is unsolved.

4

Below the source code

Binaries, firmware, HSMs and smart cards. Detecting PQC in stripped binaries has barely started.

After the session

- Re-run the lab: docker run --rm -it banklite-lab
- Run the same three views on your own code, configuration and endpoints.
- Validate every CBOM, and record what it could not see.
- Report tool bugs upstream.



github.com/preet-qclair/banklite-lab

Open-source tools used today

OpenSSL · testssl.sh · CBOMKit-theia · cdxgen · CycloneDX CLI · jq

QCLAIRVOYANCE QUANTUM LABS
Rewriting Reality With Quantum Technology